

Progetto 2: Analisi in tempo reale di dati sui voli con Apache Flink

Sistemi e Architetture per Big Data - A.A. 2025/26

Luca Carloni
0366574

Alessandro Vassallo
0366572

Andrea Galluzzi
0365026

Sommario—Il presente lavoro propone un’architettura di stream processing, incentrata sul framework Apache Flink, per l’analisi in tempo reale di un dataset di circa 2,2 milioni di eventi relativi ai voli aerei statunitensi (gennaio–aprile 2025). Il dataset storico viene trattato come uno stream mediante una fase di replay accelerato: un producer dedicato pubblica gli eventi su Apache Kafka rispettando l’ordine di event time e iniettando in modo controllato eventi fuori ordine, serializzandoli in formato Apache Avro tramite un Confluent Schema Registry. Su tale flusso vengono calcolate tre query su finestre temporali basate sull’event time, con gestione del disordine tramite watermarking. L’intera soluzione è inoltre reimplementata in Apache Spark Structured Streaming per un confronto tra i due motori. Si realizza infine la tolleranza ai guasti degli operatori con stato tramite il meccanismo di checkpointing di Flink, con verifica sperimentale del recupero dello stato. I risultati sono consegnati in formato CSV e visualizzati in tempo reale su Grafana, con due backend di storage alternativi (InfluxDB e TimescaleDB).

Index Terms—Apache Flink, Apache Kafka, Apache Avro, Confluent Schema Registry, Apache Spark, DDSketch, checkpointing, InfluxDB, TimescaleDB, Grafana

I. INTRODUZIONE

L’obiettivo del progetto è di sviluppare un’architettura di stream processing, con particolare attenzione al framework di data processing **Apache Flink**, per rispondere ad alcune query su dati storici relativi a voli aerei statunitensi, forniti dal Dipartimento dei trasporti degli Stati Uniti d’America. Il dataset contiene circa 2.200.000 eventi, che descrivono i voli effettuati nel periodo dal 1 gennaio 2025 al 30 aprile 2025. La quantità di dati forniti non giustifica appieno l’utilizzo di framework Big Data, ma le scelte progettuali e di sviluppo sono state guidate dalla volontà di simulare un contesto dove è necessario avere questo tipo di approccio. A differenza dell’analisi batch vista nel *Progetto 1: Analisi di dati sui voli con Apache Spark*, dove l’intero dataset è stato processato a posteriori, l’analisi in tempo reale richiede che gli eventi vengano elaborati man mano che vengano prodotti, mantenendo aggregazioni aggiornate su finestre temporali, gestendo possibili ritardi e disordine nell’arrivo degli eventi.

Oltre alle tre query richieste sono state anche sviluppate le estensioni previste dalla specifica: un meccanismo di rilevamento di eventi fuori ordine con il confronto di più configurazioni di watermarking e una reimplementazione dell’intera soluzione in *Apache Spark Structured Streaming* ai fini di

confronto tra i due motori di elaborazione. Inoltre, sono stati realizzati anche gli obiettivi opzionali: la dashboard real-time su Grafana e la tolleranza ai guasti tramite checkpointing.

II. ARCHITETTURA DEL SISTEMA

L’architettura realizzata è una pipeline di *stream processing* organizzata secondo una topologia ben precisa riportata in Figura 1.



Figura 1. Architettura del sistema: preprocessing, replay accelerato, ingestione su Kafka con serializzazione Avro/Schema Registry, elaborazione in streaming su Flink (e Spark come confronto), CSV, InfluxDB/TimescaleDB, Grafana.

La pipeline è organizzata nei seguenti stadi logici:

- 1) **Preprocessing.** I dataset CSV mensili vengono letti, tipizzati e arricchiti con l’*event time* di ciascun volo. Gli eventi vengono poi ordinati temporalmente e compattati in un unico file preparato, usato come sorgente del replay.
- 2) **Producer Event Replay.** Un producer dedicato riproduce il dataset storico come flusso di eventi, rispettando la temporizzazione dell’*event time* tramite un fattore di accelerazione configurabile. Il producer può inoltre introdurre eventi fuori ordine ritardandone la consegna, senza modificarne il timestamp logico, e invia marker di fine stream per drenare le ultime finestre.
- 3) **Ingestion su Apache Kafka.** Un cluster Kafka a due broker in modalità *KRaft* espone il topic *flights*, partizionato in 4 partizioni. Gli eventi sono serializzati in formato **Apache Avro** secondo il Confluent wire format, con schema registrato nel **Confluent Schema Registry**.

- 4) **Stream processing su Apache Flink.** Un cluster Flink composto da un *JobManager* e un *TaskManager* esegue le tre query con parallelismo di default pari a 4. I job consumano il topic Kafka, assegnano timestamp e watermark in *event time*, calcolano aggregazioni su finestre temporali e rendono lo stato tollerante ai guasti tramite checkpointing.
- 5) **Sink e storage dei risultati.** L'output certificato viene scritto in formato **CSV**. In parallelo, gli stessi risultati alimentano la dashboard: verso **InfluxDB** tramite topic Kafka dedicati e Telegraf, e verso **TimescaleDB** tramite sink JDBC nativo di Flink.
- 6) **Visualizzazione.** **Grafana** legge i dati da InfluxDB e TimescaleDB e mostra l'andamento temporale delle metriche prodotte dalle query.

A fini di confronto, le tre query sono state reimplementate anche in **Apache Spark Structured Streaming**, che consuma lo stesso topic Kafka e produce output CSV analoghi, così da confrontare Flink e Spark a parità di gestione.

L'intero sistema è containerizzato e orchestrato tramite **Docker Compose**, in continuità con l'approccio di deployment adottato nel anche *Progetto 1: Analisi di dati sui voli con Apache Spark*.

III. DATA INGESTION

La fase di data ingestion trasforma le sorgenti CSV grezze in uno stream di eventi ordinati e serializzati, consumabile dall'applicazione di stream processing. In particolare, questa fase si compone del preprocessing dei dati, del *producer* che effettua il replay e del trasporto su Apache Kafka con serializzazione Avro.

A. Preprocessing e dati preparati

Il dataset è fornito come collezione di file CSV, uno per ciascun mese di osservazione (da gennaio ad aprile 2025). Il preprocessing, che include la pulizia e la normalizzazione dei dati, viene eseguito in maniera disaccoppiata dal loro replay, rendendo di fatto quest'ultimo riproducibile e indipendente dal costo di parsing delle sorgenti.

1) *Selezione e tipizzazione dei campi:* Dalle sorgenti CSV vengono lette le sole 16 colonne utili alle query, scartando tutte le altre. Questi campi vengono quindi tipizzati e, a tal proposito, vogliamo porre l'attenzione sui valori numerici mancanti: la specifica consente di trattarli come *null* oppure come zero; così come nel *Progetto 1* è stata adottata la politica di lasciarli *null*, poiché un ritardo o una causa di ritardo assente è un dato *non osservato*, non un valore pari a zero. Questa politica si va a combinare con l'interpretazione di questi valori all'interno del calcolo delle query, in cui le medie ignorano automaticamente i *null* e i predicati booleani su valori *null* risultano falsi, evitando di alterare le statistiche sui ritardi.

2) *Calcolo dell'event time e ordinamento:* L'*event time* di ciascun volo è ricavato combinando i campi `YEAR`, `MONTH`, `DAY_OF_MONTH` e `CRS_DEP_TIME` (orario schedulato di partenza), come richiesto dalla specifica. Il timestamp è codificato in millisecondi dall'epoch (convenzione di Flink) e gestisce i casi limite dell'orario (ad es. il valore 2400 normalizzato a mezzanotte) e le date impossibili tramite un *fallback* al primo del mese. Il dataset viene infine *ordinato in modo stabile* per event time crescente: il replay parte quindi da una sequenza temporalmente ordinata, e l'eventuale **disordine** osservato a valle è esclusivamente *iniettato in modo controllato* dal producer, a fini sperimentali per le esecuzioni opportune, non un derivato dell'ordine originale dei file. Quindi il modulo di preprocessing genera un unico file CSV ordinato, memorizzato su un volume Docker condiviso anche con il modulo producer.

B. Apache Kafka

Il trasporto degli eventi è affidato ad **Apache Kafka**, un sistema di messaggistica distribuito basato su log append-only e partizionato. Il cluster è composto da **due broker** in modalità *KRaft*, in cui il consenso è gestito internamente da Kafka senza ricorrere ad Apache ZooKeeper: un nodo assume i ruoli di broker e controller, il secondo è broker-only e serve ad abilitare la replicazione (fattore di replicazione di default pari a 2). Gli eventi sono pubblicati sul topic `flights`, creato con **4 partizioni**: il partizionamento abilita il consumo parallelo da parte dei task di Flink e Spark. Un servizio ausiliario *Kafka UI* è incluso per il monitoraggio e il debugging del cluster.

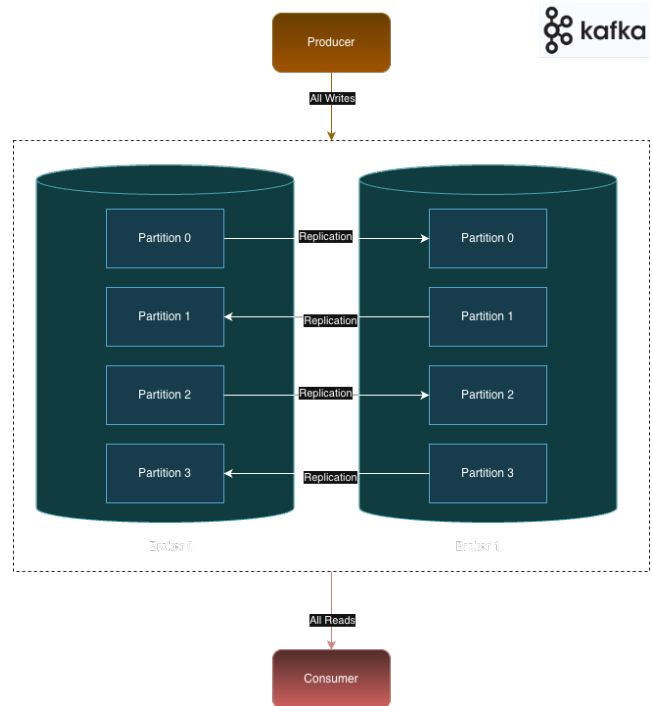


Figura 2. Kafka Cluster

C. Serializzazione Avro e Schema Registry

Gli eventi sono serializzati su Kafka in formato **Apache Avro**, un formato compatto e tipizzato, adatto allo scambio di record su uno stream. In maniera simile a quanto affrontato nel *Progetto 1: Analisi di dati sui voli con Apache Spark*, Avro è un tipo di formato basato su uno schema che definisce la tipizzazione. Nel particolare lo schema dei record `flight.avsc` va a definire la tipizzazione del messaggio `Flight`, gestendola in maniera coerente con la politica `null` del preprocessing.

Di particolare importanza la *gestione centralizzata* dello schema tramite un **Confluent Schema Registry**, un servizio che memorizza le versioni degli schemi e assegna a ciascuno un identificativo numerico. In fase di avvio, un servizio one-shot registra lo schema `flight.avsc`. I messaggi seguono il *Confluent wire format*: un *magic byte* (`0x00`), seguito dall'ID dello schema, seguito a sua volta dal *payload Avro binario*. Questo formato consente al consumer di risolvere lo schema tramite il registry e decodificare il record senza doverlo trasmettere a ogni messaggio. Il connettore `avro-confluent` di Flink e la funzione `from_avro` di Spark leggono direttamente da questo formato, risolvendo lo schema dal registry.

D. Producer

Il *producer* realizza la fase di **replay**: legge il file preparato e pubblica gli eventi sul topic `flights`, codificando ciascun record nel *Confluent wire format* descritto sopra.

1) Accelerazione temporale: Il producer applica un'accelerazione temporale che comprime in modo uniforme la scala dei tempi, preservando le relazioni temporali relative tra gli eventi: il ritardo di attesa prima dell'invio di ciascun evento è proporzionale alla distanza, in event time, dal primo evento, divisa per il fattore di accelerazione. Questo perché replicare gli eventi alla velocità originale richiederebbe l'intero arco temporale del dataset, circa 120 giorni. È stato adottato un fattore pari a **57 600×**: con esso un'ora di event time corrisponde a 0,0625 s di tempo reale, un giorno a 1,5 s e l'intero dataset viene riprodotto in circa **3 minuti**. Tale valore costituisce un compromesso fra la fedeltà alla dinamica temporale dei dati e la praticità sperimentale: il replay è abbastanza lungo da rendere osservabile il comportamento delle finestre e dei watermark in regime di streaming, ma sufficientemente rapido da permettere numerose esecuzioni ripetute durante la valutazione.

2) Iniezione del disordine (out-of-orderness): Per valutare l'impatto della completezza dei risultati sulla latenza del livello di processamento, il producer può introdurre **eventi fuori ordine** (*out-of-order*). Il disordine è simulato ritardando la consegna di una frazione di eventi, **senza mai modificarne l'event time**: un evento "trattenuto" conserva il proprio timestamp reale, ma viene inviato più tardi, comparando nel log Kafka dopo eventi a lui successivi.

Operativamente, con una certa probabilità p un evento viene posto in un *heap* ordinato per istante di consegna pianificato (in ritardo rispetto all'event time originale) e rilasciato solo quando il tempo di replay raggiunge tale istante. L'entità del ritardo è campionata da una distribuzione configurabile, *uniforme* oppure *esponenziale*, con un tetto massimo definito. Quest'ultimo è espresso in secondi di event time e rappresenta il *bound di out-of-orderness* D con cui confrontare il watermark: se il ritardo ammesso dal watermark è $\geq D$, gli eventi ritardati rientrano comunque nella finestra corretta; in caso contrario alcuni arrivano troppo tardi e vengono scartati come *late*. Per garantire la riproducibilità degli esperimenti il campionamento è governato da un seed fisso, e il parametro `max_in_flight_requests_per_connection` è posto a 1 per preservare l'ordine relativo degli eventi non trattenuti.

3) Chiusura dello stream: Poiché lo stream è finito, trattandosi di un replay di un dataset costruito precedentemente, l'ultima finestra event time non verrebbe chiusa spontaneamente in assenza di nuovi eventi che ne facciano avanzare il watermark. Per drenarla in modo automatico, al termine del replay il producer emette su *ogni* partizione del topic un *marker* di fine stream con event time collocato molto nel futuro rispetto all'anno di riferimento del dataset: abbiamo optato per l'anno 2200. Ciò spinge tutti i watermark per-partizione oltre l'ultima finestra reale, che viene così calcolata ed emessa senza ricorrere al comando manuale di drenaggio di Flink. Il marker è identificato dal codice compagnia speciale `__EOS__` ed è quindi escluso da tutte le query. In alternativa, l'arresto ordinato tramite il drenaggio del job produce il medesimo effetto.

IV. STREAM PROCESSING CON APACHE FLINK

A. Apache Flink

Apache Flink è un motore di elaborazione distribuita di stream, che tratta i dati come flussi illimitati elaborati a bassa latenza con stato gestito e tolleranza ai guasti. Il cluster utilizzato è composto da un *JobManager*, che coordina il submit e la schedulazione dei job, e da un *TaskManager* che ne esegue i task con un livello di parallelismo pari a 4. Flink distingue il *tempo di elaborazione* dall'*event time*: le query di questo progetto operano interamente in event time, ricavato dal timestamp logico assegnato a ciascun evento, così da produrre risultati deterministici e indipendenti dalla velocità di arrivo. L'avanzamento del tempo è tracciato dai *watermark*: un watermark di valore t dichiara che non si attendono più eventi con timestamp inferiore a t , e autorizza la chiusura delle finestre il cui bordo è stato superato. La soglia di ritardo ammissibile del watermark rispetto al fronte dei dati costituisce il budget di attesa massima per gli eventi fuori ordine. Il checkpointing periodico garantisce la consistenza dello stato in caso di guasto.

I job sfruttano sia la *Table API/SQL*, di più alto livello e ottimizzata dal planner, sia la *DataStream API*, di più basso livello. Le tre query adottano un approccio ibrido, combinando le due API secondo l'operazione più naturale in ciascuno stadio.

1) *Eventi fuori ordine e watermarking*: È stato implementato un meccanismo di rilevamento di eventi fuori ordine e si confrontano più configurazioni di watermarking. Il disordine è generato dal producer ritardando la consegna di una quota di eventi entro un **limite massimo** D , come già introdotto nella sezione relativa al producer. La vera manopola che governa il compromesso tra *completezza* dei risultati e *latenza* del livello di processamento è il **ritardo massimo ammesso dal watermark** d . Un d ampio attende più a lungo gli eventi fuori ordine (completezza elevata, latenza alta), mentre un d ridotto chiude prima le finestre ma rischia di scartare eventi in ritardo (latenza bassa, possibile perdita). Sotto disordine uniforme in $[0, D]$ e trascurando i gap di quantizzazione del watermark, la frazione attesa di eventi persi in una finestra di ampiezza W vale $\text{loss} = p(D - d)^2 / (2DW)$: nulla quando $d \geq D$ (configurazione *safe*), massima quando $d = 0$ (configurazione *aggressive*).

Due elementi influenzano la *quantizzazione* con cui il watermark viene effettivamente applicato:

- l'intervallo di emissione periodica del watermark stesso (auto-watermark-interval)
- Per gli operatori Python, la durata del *bundle* (`python.fn-execution.bundle.time`), poiché i watermark sono applicati solo al confine del bundle.

Col valore di default di questi due parametri di configurazione, l'accelerazione dilaterrebbe a tal punto il loro equivalente in event-time da gonfiare il ritardo con cui avanza il watermark, permettendo anche a eventi arrivati in ritardo di entrare all'interno della finestra stessa. Infatti, i valori default (dell'ordine di 200 ms per l'emissione periodica del watermark e 1 s per il bundle time) e un fattore di accelerazione $57.600\times$ si traducono in salti enormi di event-time (200 ms $\rightarrow$ 3,2 h, 1 s di bundle $\rightarrow$ 16 h). Tra un salto e l'altro il watermark resta indietro per lunghi tratti di event-time, e finché non lo supera la finestra non scatta e non viene chiusa. Il risultato è che un evento fuori ordine trova ancora aperta la propria finestra e viene incluso, invece di essere scartato. Quindi negli esperimenti di completezza entrambi sono ridotti per rendere trascurabile il gap virtuale introdotto e isolare l'effetto di d . Il numero di eventi scartati perché arrivati oltre il watermark è misurato direttamente dalla metrica Flink `numLateRecordsDropped`, raccolta per job via REST API.

2) *Tolleranza ai guasti e Checkpointing*: Per la gestione della tolleranza ai guasti degli operatori con stato mediante il meccanismo di *checkpointing* di Flink, a intervalli regolari (10 s) Flink esegue uno snapshot coerente (*barrier-aligned*) dello stato di *tutti* gli operatori del job insieme agli offset

Kafka della sorgente; in caso di guasto, il job viene riavviato e ripristinato dall'ultimo checkpoint completato, mentre la sorgente Kafka riparte dagli offset salvati, senza perdita né duplicazione nei risultati.

Il meccanismo è applicato a tutti i job Flink:

- Modalità **EXACTLY_ONCE** (barrier allineate) con `max_concurrent=1`;
- Pausa minima tra checkpoint (5 s) per garantire progresso all'elaborazione, timeout di checkpoint (60 s) e numero di fallimenti consecutivi tollerati (3) prima di far fallire il job Flink;
- Checkpoint **esternalizzati**: l'ultimo checkpoint sopravvive a cancellazione/guasto ed è riutilizzabile per il recovery (`RETAIN_ON_CANCELLATION`);
- *Restart strategy* a ritardo fisso (fino a 10 tentativi, 10 s di attesa ciascuno): senza di essa il job fallirebbe invece di ripristinarsi.

3) *Query Q1*: La query Q1 realizza il monitoraggio in tempo reale dello stato operativo delle principali compagnie aeree. Facendo riferimento alle sole compagnie aeree **AA** (American Airlines), **DL** (Delta), **UA** (United) e **WN** (Southwest), gli eventi sono aggregati su *finestre tumbling di durata 1 ora basate sull'event time*. Per ciascuna finestra e per ciascuna compagnia la query calcola:

- il numero totale di voli osservati
- il numero di voli completati (non cancellati e non deviati), cancellati e deviati
- il valor medio di `DEP_DELAY` sui soli voli non cancellati
- il tasso di cancellazione (percentuale di voli cancellati sul totale della finestra)
- e il tasso di partenze in ritardo (percentuale di voli non cancellati con `DEP_DELAY` maggiore di 15 minuti).

La query è espressa con la *Table API/SQL*. Le compagnie di interesse sono filtrate *prima* del windowing, così che Flink non assegni alle finestre eventi di compagnie irrilevanti. L'aggregazione utilizza la *windowing TVF* `TUMBLE` su finestre di un'ora e raggruppa per finestra e compagnia. La gestione dei valori mancanti è coerente con la politica `null: COALESCE(cancelled, 0)` (e analogamente `diverted`) interpreta lo stato assente come "non cancellato/non deviato", mentre `AVG(dep_delay)` ignora automaticamente le righe con ritardo `null`. Il tasso di cancellazione è calcolato su *tutti* i voli della finestra, mentre il tasso di partenze in ritardo è normalizzato sui soli voli non cancellati, come da specifica.

L'aggregazione è materializzata in un'unica vista da cui leggono sia il sink CSV che i sink della dashboard leggono tutti la stessa vista.

4) *Query Q2*: La query Q2 identifica in tempo reale gli aeroporti di partenza maggiormente interessati da ritardi significativi. Considerando tutti i voli non cancellati e non deviati, un volo presenta un *ritardo significativo* se `DEP_DELAY`

è maggiore di 30 minuti. La query è calcolata su tre finestre event-time: **1 ora, 6 ore e dall'inizio del dataset**. Per ciascuna finestra si produce la classifica aggiornata dei primi 10 aeroporti per numero decrescente di voli con ritardo significativo, considerando solo gli aeroporti con almeno 30 voli non cancellati e non devianti.

La query è espressa *interamente con la Table API/SQL* e, per ciascuna finestra temporale, si sviluppa in due momenti logici: prima si costruisce una sintesi dei ritardi in partenza scalo per scalo, poi su questa sintesi si stila la classifica degli aeroporti più critici.

- **Stadio 1 – sintesi per aeroporto.** L'analisi considera i soli voli effettivamente operati, scartando quelli cancellati o dirottati per non falsare le statistiche con partenze mai avvenute. I voli superstiti vengono raggruppati per finestra e per aeroporto di origine e, per ogni gruppo, si ricavano alcune misure di sintesi: il numero complessivo di partenze, quante di esse abbiano accumulato un ritardo rilevante (oltre la mezz'ora), il ritardo medio e quello massimo. Per evitare che scali con pochissimo traffico entrino in graduatoria con statistiche poco affidabili, si conservano soltanto gli aeroporti con almeno trenta voli nella finestra. Accanto ai valori aggregati, il sistema raccoglie anche l'elenco dei voli in ritardo e ne trattiene, ordinati dal più grave, i venti casi peggiori: un dettaglio che aiuta il lettore a riconoscere quali collegamenti abbiano pesato di più sul risultato.
- **Stadio 2 – classifica top-10.** Sulla sintesi così ottenuta si ordina, all'interno di ciascuna finestra, la graduatoria degli aeroporti. Il criterio privilegia anzitutto il numero di voli gravemente in ritardo; a parità di questi interviene il ritardo medio e, come ultimo discriminante, l'identificativo dell'aeroporto, così da rendere l'ordinamento univoco e riproducibile. Di ogni finestra si trattengono i primi dieci scali. Poiché la classifica di una finestra viene prodotta una sola volta, quando essa è ormai definitivamente chiusa, il flusso in uscita è puramente incrementale e può quindi essere scritto direttamente sul file CSV.

Un discorso a parte merita la finestra "dall'inizio del dataset". In assenza di un costrutto nativo per una finestra illimitata, la si è realizzata con una finestra fissa sufficientemente ampia (365 giorni) entro cui l'intero periodo osservato (gennaio–aprile 2025) ricade in un unico intervallo; questo si chiude soltanto quando il marker di fine flusso (EOS) segnala che non arriveranno altri dati e spinge il tempo interno oltre il suo bordo. Infine, come già in Q1, tutte le scritture in uscita verso il sink CSV e i sink verso la dashboard sono raccolte in un unico blocco, in modo che la sorgente venga letta una sola volta e alimenti insieme tutti i destinatari.

5) *Query Q3:* La query Q3 calcola in tempo reale la distribuzione dei ritardi in partenza per compagnia e fascia oraria. Facendo riferimento alle compagnie aeree AA, DL, UA e WN e ai soli voli non cancellati e non devianti con `DEP_DELAY`

presente, per ciascuna compagnia e per ciascuna delle 24 fasce orarie (ricavate da `CRS_DEP_TIME`) si calcolano il numero di voli, i percentili 25/50/75/90, il minimo e il massimo di `DEP_DELAY`. La query è calcolata su tre finestre event-time: **1 giorno, 7 giorni e dall'inizio del dataset**.

Il job combina le due API come Q1/Q2: la sorgente Kafka è definita come tabella e convertita in un `DataStream` su cui avviene la computazione in `DataStream API`. In questo caso si ha la finestra con `AggregateFunction` il cui accumulatore è un `DDSketch`, di cui è possibile trovare una spiegazione più completa nella sezione successiva. Le scritture in uscita avvengono lo stesso pattern visto per le precedenti query, verso CSV e dashboard.

6) *Stima approssimata dei percentili con DDSketch in Q3:* La specifica richiede di calcolare i percentili dei ritardi alla partenza (`DEP_DELAY`) in tempo reale, senza ordinare né accumulare interamente i valori, mediante un algoritmo approssimato. È stato adottato **DDSketch** [1], uno sketch dei quantili con garanzia di *errore relativo massimo garantito* α , implementato in Python puro. L'asse dei valori è partizionato in bucket a spaziatura logaritmica: questo vuol dire che l'algoritmo mappa un qualunque valore $v > 0$ nel bucket di indice $\lceil \log_{\gamma} v \rceil$, dove $\gamma = (1 + \alpha)/(1 - \alpha)$ è il ratio del bucket, e ogni bucket mantiene solo un contatore intero. Per una query di quantile mappata al bucket di indice i , il valore restituito è il rappresentante

$$\tilde{v} = \frac{2\gamma^i}{\gamma + 1},$$

che garantisce un errore relativo $|\tilde{v} - v|/v \leq \alpha$ su tutto l'intervallo $(\gamma^{i-1}, \gamma^i]$ coperto dal bucket. Il quantile q si ottiene scorrendo i contatori in ordine crescente di valore fino alla posizione $q(n - 1)$.

Poiché i ritardi possono essere negativi (voli in anticipo), si usano due store speculari (uno per i valori positivi e uno per i valori negativi mappati su $|v|$) più un contatore per gli zeri. Minimo, massimo e conteggio sono mantenuti *esatti*, come richiesto. Due sketch si fondono sommando i contatori omologhi *fully mergeable*, proprietà che li rende compatibili con la `merge()` delle finestre di Flink.

V. STREAM PROCESSING CON APACHE SPARK STRUCTURED STREAMING

A. Apache Spark Structured Streaming

A fini di confronto tra motori, le tre query sono state reimplementate in **Apache Spark Structured Streaming**, che consuma lo stesso topic Kafka e ne decodifica i messaggi, rimuovendo i byte relativi all'intestazione del *Confluent wire format*. Ogni query adotta lo stesso modello di finestre event-time delle controparti Flink, dichiarate con `F.window` e con `withWatermark` per la gestione del disordine. Essendo il replay finito, un controllo di inattività arresta automaticamente i job Spark quando non arrivano più dati da Kafka per un intervallo configurato.

VI. STORAGE DELLE METRICHE: INFLUXDB E TIMESCALEDB

Oltre all'output CSV, l'architettura prevede la persistenza delle metriche delle query su un sistema di storage che funge anche da sorgente per la visualizzazione. Sono stati implementate due backend alternativi e indipendenti, mantenuti entrambi a fini di confronto. I due backend sono semplici *consumatori paralleli* delle medesime viste dei job, dunque non sostituiscono né possono contraddire l'output consegnato. È previsto un topic/tabella per ciascuna finestra di ogni query.

A. InfluxDB

InfluxDB è un database *time-series nativo*. Non disponendo di un connettore Flink diretto, l'integrazione avviene tramite un ponte: il job pubblica le righe su un topic Kafka dedicato da cui **Telegraf** le consuma e le scrive su InfluxDB.

B. TimescaleDB

TimescaleDB è un database relazionale *SQL* (PostgreSQL con estensione time-series). A differenza di InfluxDB dispone di un *sink JDBC nativo* di Flink, che scrive direttamente senza componenti intermedi.

C. Confronto

Tabella I
CONFRONTO TRA I DUE BACKEND DI STORAGE DELLE METRICHE.

	InfluxDB	TimescaleDB
Paradigma	time-series nativo	SQL + estensione TS
Connettore Flink	assente (Telegraf)	JDBC nativo
Componenti	Kafka+Telegraf+DB	solo DB
Idempotenza	last()	upsert su PK
Linguaggio Query	Flux	SQL

VII. VISUALIZZAZIONE: GRAFANA

La visualizzazione real-time delle metriche prodotte dalle query è affidata a **Grafana**, in continuità con la scelta adottata nel *Progetto 1: Analisi di dati sui voli con Apache Spark*. Grafana è condiviso fra i due backend: tramite *provisioning* dichiarativo configura automaticamente entrambi i *datasource* (InfluxDB con linguaggio Flux e TimescaleDB con SQL) e le rispettive dashboard per le tre query, così da poter confrontare le due soluzioni fianco a fianco durante la presentazione. I pannelli mostrano l'andamento temporale, finestra dopo finestra, delle metriche delle query. Poiché il timestamp dei punti è l'event time della finestra, l'intervallo temporale delle dashboard è impostato sul periodo del dataset (gennaio–aprile 2025): durante il replay accelerato le curve si costruiscono progressivamente, dando evidenza visiva della natura streaming dell'elaborazione.

Per quanto riguarda la finestra globale, definita dall'inizio del dataset, abbiamo interpretato la finestra come dall'inizio alla fine del dataset, chiudendola una volta arrivato l'ultimo evento. Ciò significa che nella finestra globale l'evoluzione real-time delle metriche non può essere visualizzato, dato che

essa emette il proprio risultato solo quando il watermark ne supera il bordo destro: la chiusura è innescata dal *marker di fine stream* iniettato dal producer, che porta il watermark oltre il confine della finestra.

Per completezza abbiamo anche implementato una *vista cumulativa live* a bassa latenza con snapshot intermedi, in modo da vedere l'evoluzione real-time delle metriche. La cadenza di emissione è regolata in event time in Q2 (un'istantanea ogni 45 minuti di event-time) e in tempo di elaborazione in Q3 (ogni 15 ore). La vista cumulativa calcola dunque la stessa quantità della finestra globale, ma la espone come serie temporale che ne mostra l'evoluzione *real-time*, mentre la finestra globale ne fornisce il valore finale, certificato a stream concluso.

VIII. DEPLOYMENT

Il deployment è stato effettuato tramite containerizzazione dei nodi del sistema con **Docker**, simulando un cluster di macchine distinte. Ogni nodo dell'architettura corrisponde a un container collegato a una stessa Docker Network, e l'orchestrazione dell'intero cluster è gestita tramite **Docker Compose**: un file descrive in modo dichiarativo tutti i servizi (i due broker Kafka, lo Schema Registry, JobManager e TaskManager Flink, i job delle query, i job Spark, i backend di dashboard e Grafana), definendone i parametri di configurazione e la comunicazione tra i nodi. La configurazione applicativa è centralizzata in un file YAML montato nei container, che consente di variare i parametri (senza ricostruire le immagini). Gli esperimenti invece sono descritti da file di override che ne estendono la configurazione base.

Come nel *Progetto 1: Analisi di dati sui voli con Apache Spark*, eseguire tutti i container sulla stessa macchina porta i nodi a contendersi un insieme ristretto di risorse computazionali e ad avere latenze di comunicazione prossime allo zero, condizione lontana da un caso reale distribuito ma adeguata agli scopi didattici e di valutazione comparativa del progetto.

IX. VALUTAZIONE DEI RISULTATI

Sono stati valutati sperimentalmente:

- L'impatto del parallelismo dell'applicazione a fronte della latenza di processing
- L'impatto della completezza dei risultati rispetto alla latenza introducendo *out-of-orderness*, in un contesto di rilevamento degli eventi fuori ordine, gestito tramite watermarking
- Confronto delle prestazioni tra Apache Flink e Apache Spark Structured Streaming

A tal fine sono stati predisposti un insieme di *esperimenti* riproducibili e degli *strumenti* per raccogliere le metriche: i valori numerici, le tabelle e i grafici sono prodotti eseguendo tali esperimenti e raccogliendone le metriche tramite i suddetti strumenti. Oltre alla discussione sperimentale, in questa sezione discutiamo anche dei risultati delle query.

A. Strumenti di misura

Un runner unico (`scripts/run_experiment.ps1`) esegue una delle query su un certo motore (Flink o Spark), applicando una configurazione di esperimento. Le metriche sono raccolte da script dedicati che scrivono i risultati in diversi file CSV:

- **Strumento di misura delle Performance:** campiona via REST le metriche del job Flink, come l'*active time*, e ne ricava il *throughput* del vertice sorgente (in record/s) e le percentuali di *busy* e di *backpressure* dell'operatore più carico (per individuare il collo di bottiglia).
- **Strumento di misura della Completezza:** somma la metrica `numLateRecordsDropped` degli operatori finestra, ossia gli eventi scartati perché giunti oltre il watermark. In Q3 i record in ritardo vengono deviati a un *side output* omonimo alla metrica nativa di Flink, poiché a causa dell'utilizzo di DDSketch, il contatore nativo `numLateRecordsDropped` dell'operatore finestra resta a 0.

B. Esperimenti predisposti

Gli esperimenti descrivono lo scenario per quanto riguarda disordine, watermark, e parallelismo.

- **Baseline:** fornisce l'output di riferimento per il controllo di completezza, senza iniettare disordine.
- **Sweep di watermarking:** disordine uniforme con **limite massimo di ritardo D** pari all'**ampiezza della finestra W** della query, variando **solo il ritardo ammesso d** tra *aggressive* ($d = 0$), *intermedio* ($d = W/2$) e *safe* ($d = W$). Le configurazioni sono organizzate per finestra così da sondare il compromesso completezza/latenza dove risiedono le finestre di ciascuna query.
- **Sweep di parallelismo** (`perf_par1`, `perf_par2`, `perf_par4`): scenario baseline tenuto fisso, variando solo il parallelismo di Flink tra 1, 2 e 4 (numero di partizioni del topic), per isolarne l'effetto su throughput e latenza.

Tabella II

FINESTRE E QUERY CORRISPONDENTI. PER CIASCUNA FINESTRA $D = W$ E IL RITARDO AMMESSO d È VARIATO TRA 0 (*aggressive*), $W/2$ E W (*safe*).

Finestra	W [s]	d testati [s]	Query (finestra)
1h	3600	0 / 1800 / 3600	Q1, Q2 (1h)
6h	21600	0 / 10800 / 21600	Q2 (6h)
1d	86400	0 / 43200 / 86400	Q3 (1d)
7d	604800	0 / 302400 / 604800	Q3 (7d)

C. Confronto Apache Flink e Apache Spark Structured Streaming

Il confronto **Flink vs Spark** si ottiene eseguendo lo stesso scenario con i due motori.

D. Checkpointing

La prova di **checkpointing** serve a fornire una verifica del corretto recupero dello stato a seguito di un guasto. Il recupero è verificato da uno script dedicato che riproduce uno scenario di *guasto di nodo*:

- Con il job Q1 in esecuzione e il producer avviato (rallentato, così che il guasto cada durante l'elaborazione), lo script attende via REST il completamento di almeno un checkpoint
- Registra il valore di `numRestarts` come baseline
- Quindi **uccide il container del TaskManager**, simulando un guasto
- Riavvia poi il TaskManager e ne attende la registrazione
- L'esito è considerato positivo se:
 - il job riporta un ripristino dello stato da checkpoint: il campo `latest.restored` viene valorizzato, con l'ID del checkpoint da cui è stato recuperato lo stato
 - e il contatore `numRestarts` è aumentato, evidenza dell'avvenuto riavvio

Lo script prevede anche una modalità *baseline* senza guasto, il cui output può essere confrontato con quello del run con guasto per verificare che i risultati coincidano (e quindi non ci sia stata alcuna perdita né duplicazione). Inoltre, ciò permette un confronto della latenza di processamento con e senza guasto.

E. Discussione dei risultati

1) **Completezza vs Latenza (watermarking):** In questo studio valutiamo la completezza e latenza al variare di d , la soglia di ritardo ammessa dal watermark. Si hanno i seguenti parametri:

- Percentuale $p = 0.3$ di record a cui viene applicato un ritardo
- Ritardo $D = W$ con W dimensione della finestra, che dipende dalla specifica per ciascuna query
- Si noti che la percentuale di record persi è calcolata sulla popolazione di riferimento, che per Q1 e Q3 è pari $\approx 1.2M$ dato che considerano solo un sottoinsieme di compagnie aeree, mentre per Q2 è pari a $\approx 2.2M$
- La perdita attesa è stata calcolata con la formula $p(D - d)^2 / (2DW)$, con $D = W$. In realtà la perdita attesa è il **limite worst-case**. È possibile che la perdita misurata resti al di sotto di tale limite a causa di uno slack dovuto alla quantizzazione del watermarking e alla densità non uniforme del traffico.
- In questo caso abbiamo volutamente limitato il parallelismo, con **1 singola partizione e 1 solo task Flink**. Inoltre, **l'aggregazione è stata limitata a una fase sola**. Entrambe le scelte sono state prese consapevolmente con un unico scopo: ottenere una lettura pulita e priva di rumore del contatore `numLateRecordsDropped`, una metrica che Flink calcola internamente sull'operatore di windowing e che conteggia i record scartati perché arrivati in ritardo oltre il ritardo consentito, a cui si è

spesso fatto riferimento durante la relazione con d . In particolare:

- **Parallelismo unitario (1 partizione, 1 solo task Flink).** La metrica è esposta per singolo *subtask*: con più istanze parallele il watermark che determina il ritardo su un operatore è il minimo fra tutti i canali di ingresso. L'insieme dei record scartati dipenderebbe quindi dall'*interleaving* fra le partizioni, rendendo la misura non deterministica e variabile fra un'esecuzione e l'altra. Con una singola partizione e un singolo task si ha invece un'unica traiettoria deterministica del watermark e un unico contatore, dunque una misura **ripetibile**.
- **Aggregazione a fase singola.** Il contatore `numLateRecordsDropped` è registrato solo sull'operatore di windowing che effettua il controllo del ritardo rispetto al watermark. Quando è attiva l'ottimizzazione a due fasi, la pre-aggregazione locale (operatore `LocalWindowAggregate`, a runtime `LocalSlicingWindowAggOperator`) non espone questa metrica e collassa i record grezzi in *slice* parzialmente aggregate: quindi a valle di questa ottimizzazione il conteggio non rifletterebbe più il numero di record di *input* in ritardo, bensì quello delle *slices*. Forzando un'unica fase, ogni record attraversa l'unico operatore che sia effettua il controllo del ritardo, sia espone il relativo contatore, garantendo un conteggio esatto.

Si noti che all'aumentare della soglia di ritardo ammesso dal watermark d , che corrisponde al ritardo di watermark, la latenza che viene introdotta è un **latenza di emissione dei risultati pari proprio a d** , non un ritardo a livello di elaborazione, dato che la variazione del ritardo di watermark d non incide sul costo computazionale delle query. In tutte le configurazioni il numero di record ingeriti dalla sorgente rimane invariato e, a meno del rumore sperimentale, altrettanto avviene per il *throughput* medio e per il tempo di sola elaborazione. *A mutare è esclusivamente il numero di record tardivi scartati*. Infatti il watermark rappresenta una soglia di avanzamento logico che segue il massimo timestamp osservato con uno scarto pari a d : incrementare d significa soltanto posticipare l'istante (in *event-time*) in cui una finestra $[a, b)$ è ritenuta completa e viene quindi emessa, poiché essa chiude unicamente all'osservazione di un evento con timestamp $\geq b + d$. Il motore, tuttavia, non sospende l'elaborazione in attesa dei record tardivi: continua a consumare il flusso alla medesima velocità, processando nel frattempo i record successivi, i quali appartengono a finestre temporalmente più avanzate e sarebbero comunque letti. L'attesa dei tardivi non introduce dunque né una pausa misurabile in *wall-clock* né un carico di lavoro aggiuntivo. Coerentemente, una finestra mantenuta aperta più a lungo comporta un maggiore costo di *stato*, perché l'accumulatore permane in memoria per un intervallo superiore, ma non un maggiore costo di *calcolo*. Se ne conclude che il prezzo della completezza non si paga in tempo di elaborazione, ma

esclusivamente in latenza di emissione, pari per costruzione a d : costo computazionale e ritardo del risultato misurano aspetti ortogonali del sistema e non vanno confusi.

2) *Confronto Flink vs Spark*: Il confronto tra Apache Flink e Apache Spark Structured Streaming è condotto sulla medesima piattaforma e a parità di condizioni: entrambi i motori consumano lo stesso topic Kafka con 4 partizioni e un livello di parallelismo pari a 4, drenando l'intero backlog pre-caricato, al fine di valutare e confrontare solamente le prestazioni dei due motori di processamento. Prima di confrontarne le prestazioni occorre verificare che le due implementazioni producano gli *stessi* risultati e la Tabella V riporta proprio la cross-validation che abbiamo eseguito riga per riga dell'output in CSV. Per Q1 e Q2 l'output coincide esattamente su tutte le colonne. Per Q3, che stima i percentili in modo approssimativo, il conteggio, il minimo e il massimo sono esatti. Accertata l'equivalenza funzionale, il confronto si riduce alla sola dimensione prestazionale, riassunta in Tabella IV.

Query	W	d [s] (event-time)	Regime	LateDropped [record]	Record persi sulla popolazione di riferimento [%]	Record persi attesi [%]
Q1	1h	0	aggressivo	143 893	10.9	15.00
		1800	intermedio	25 513	1.93	3.75
		3600	completo	0	0.00	0.00
Q2	1h	0	aggressivo	241 324	10.82	15.00
		1800	intermedio	44 575	2.00	3.75
		3600	completo	0	0.00	0.00
Q2	6h	0	aggressivo	288 977	12.96	15.00
		10800	intermedio	65 428	2.93	3.75
		21600	completo	0	0.00	0.00
Q3	1d	0	aggressivo	139 790	10.59	15.00
		43200	intermedio	35 032	2.65	3.75
		86400	completo	0	0.00	0.00
Q3	7d	0	aggressivo	185 587	14.05	15.00
		302400	intermedio	29 559	2.24	3.75
		604800	completo	0	0.00	0.00

Tabella III

COMPLETEZZA SOTTO OUT-OF-ORDERNESS AL VARIARE DEL WATERMARK DELAY d

Query	Flink (p4/n4)		Spark (p4/n4)	
	Throughput [rec/s]	Elapsed time [ms]	Throughput [rec/s]	Elapsed time [ms]
Q1 (1h)	386665	5766	336521	6625
Q2 (1h)	226478	9844	189097	11790
Q3 (1d)	26369	84547	211062	10563

Tabella IV

CONFRONTO PRESTAZIONI FLINK VS SPARK (BACKLOG PRE-CARICATO, $p=n=4$).

Tabella V

CROSS-VALIDATION DELL'OUTPUT CERTIFICATO (CSV): CONFRONTO RIGA PER RIGA TRA LE IMPLEMENTAZIONI FLINK E SPARK.

Query / finestra	Righe (Flink / Spark)	Esito
Q1 - 1h	9765 / 9765	=
Q2 - 1h	14 265 / 14 265	=
Q2 - 6h	3601 / 3601	=
Q2 - global	10 / 10	=
Q3 - 1d	9763 / 9763	≈
Q3 - 7d	1475 / 1475	≈
Q3 - global	84 / 84	≈

= output identico su tutte le colonne.

≈ count, min, max esatti; percentili concordi entro l'errore di approssimazione dei due sketch.

I dati delineano due comportamenti distinti a seconda delle API impiegate.

- **Q1 e Q2 - Table API/SQL su JVM.** Sulle query puramente relazionali i due motori sono sostanzialmente allineati, ma con un lieve vantaggio di Flink: infatti il throughput passa da 336 521 rec/s a 386 665 rec/s su Q1 (con un incremento del +15% circa) e da 189 097 a 226 478 rec/s su Q2 (con un incremento del +20%

circa), con tempi di elaborazione corrispondentemente inferiori. Eseguite interamente in JVM e ottimizzate dal planner, queste query non evidenziano alcuno svantaggio strutturale di Flink rispetto al modello a micro-batch di Spark.

- **Q3 - DataStream API con PyFlink.** In questo caso il quadro si ribalta nettamente, dato che Spark completa l'elaborazione in 10.6 s contro gli 84.5 s di Flink, con un throughput di 221 1062 contro 26 369 rec/s, ovvero un fattore di circa $9.2\times$ a favore di Spark. Il divario *non* dipende dall'algoritmo di stima dei quantili: tenuto fisso l'algoritmo, la differenza è quindi attribuibile per intero al modello di esecuzione del codice Python. Q3 è infatti l'unica query realizzata in Python, tramite la DataStream API di PyFlink, dove ogni record deve essere serializzato e trasferito tra la JVM e il processo Python (e il risultato ricondotto indietro): tale costo per-record domina l'aggiornamento del DDSketch, di per sé economico. Spark, al contrario, elabora il codice Python su blocchi di record, ammortizzando l'attraversamento del confine e mantenendo prestazioni prossime a quelle della versione nativa. È dunque l'overhead di esecuzione di PyFlink, e non il DDSketch, a determinare il crollo del throughput di Flink su Q3.

In sintesi, a parità di risultati Flink e Spark si equivalgono sulle query relazionali - con un margine a favore di Flink - mentre l'unico distacco marcato, quello di Q3, misura il costo del *bridge* Python di PyFlink e non una carenza architetturale del motore.

3) *Impatto del recupero da checkpoint:* Per valutare l'impatto del recupero da checkpoint è stato eseguito un confronto tra

due tipologie di esperimenti sulla query Q1, entrambe con producer accelerato a $57600\times$: nel primo esperimento sono state eseguite 10 esecuzioni indipendenti in cui si è atteso il completamento di almeno un checkpoint prima dell'iniezione di guasto simulato, mentre nel secondo esperimento per le 10 esecuzioni nelle stesse condizione non è stato introdotto alcun guasto, per comparazione di risultati e statistiche di completamento. La misura considera il tempo dal lancio del producer fino al consolidamento delle finestre finali, escludendo setup Docker, reset del topic, preprocessing, submit del job, merge e cleanup.

I valori riportati sono ottenuti come media di 10 esecuzioni indipendenti. Mediamente, l'esecuzione senza guasto ha richiesto 200.228 s, mentre quella con guasto ha richiesto 200.346 s. L'iniezione del guasto è avvenuta in media a 7.916 s dall'inizio dell'esecuzione; Flink ha rilevato il fallimento a 52.620 s e il ripristino da checkpoint è stato osservato a 61.669 s. Il tempo di recupero medio misurato, dall'uccisione del TaskManager al ripristino del job, è quindi pari a 53.753 s. In tutte le esecuzioni la metrica `numRestarts` è passata da 0 a 1 e la verifica è terminata con esito positivo.

Tabella VI

TEMPI MEDI SU 10 ESECUZIONI PER LA QUERY Q1 (*acceleration factor* = 57600). IL TEMPO DI *recovery* È DEFINITO SOLO IN PRESENZA DI FAULT.

Modalità	Execution (ms)	Producer (ms)	Recovery (ms)
Con fault	200 346.4	185 335.3	53 752.5
Senza fault	200 228.4	185 218.4	–

Il guasto introduce quindi un aumento del tempo end-to-end di appena circa 0,06% rispetto al run senza guasto, un valore che è giustificato dalla semplice variabilità fra le esecuzioni. Pur essendo il tempo di recupero osservato di circa 54 s, il suo impatto sul tempo complessivo della pipeline è trascurabile: il run con guasto termina in media solo circa 0,12 s dopo quello senza guasto, perché durante il ripristino il producer continua a pubblicare eventi su Kafka e Flink, una volta riavviato, riprende dagli offset salvati nell'ultimo checkpoint completato e recupera il backlog accumulato. Il costo del fault recovery si manifesta dunque come una pausa temporanea dell'elaborazione, riassorbita entro la finestra di esecuzione senza allungare in modo apprezzabile il tempo end-to-end.

4) *Studio del parallelismo*: Per valutare la scalabilità delle tre query su Flink sono stati misurati il throughput medio e il tempo di elaborazione al variare di due parametri fatti crescere congiuntamente: il grado di parallelismo degli operatori in Flink (p) e il numero di partizioni del topic Kafka di ingresso (n), seguendo una configurazione $n = p$ che varia da 1 a 4. I risultati, riassunti nella Tabella e nella Figura seguenti, mettono in luce due regimi nettamente distinti, resi evidenti soprattutto dall'andamento del tempo medio di occupazione degli operatori (*busy time*).

Il primo regime è quello *compute-bound*, rappresentato dalla sola Q3. In questo caso gli operatori restano costantemente

saturi, dato che si ha busy medio tra l'88,9% e il 94,1%, stabilmente al di sopra della soglia dell'85%, abbastanza alta. Inoltre l'incremento del parallelismo si traduce direttamente in un aumento sostanziale del throughput, con uno speedup di $2,41\times$ passando da $p = 1$ a $p = 4$. In questo caso il parallelismo costituisce la leva più efficace. Il secondo regime che consideriamo è quello *ingestion-bound*, cui appartengono Q1 e Q2: il tempo di occupazione degli operatori resta lontano dalla saturazione (per esempio si può osservare come per Q1 addirittura decresca dal 42,2% al 24,0% all'aumentare di p , mentre per Q2 si mantiene su valori poco più alti) segnalando che il collo di bottiglia non risiede nella capacità di calcolo. Sebbene il throughput cresca comunque (speedup di $1,48\times$ per Q1 e $1,52\times$ per Q2), tale miglioramento è probabilmente dato dal contributo dell'aumentato numero di partizioni n del topic di Kafka (che come abbiamo detto cresce in parallelo a p) più che all'incremento del parallelismo di calcolo, come confermato dalla progressiva sotto-utilizzazione degli operatori. Per queste query, dunque, il margine di miglioramento non passa dall'aggiunta di parallelismo di elaborazione, quanto piuttosto da interventi sulla capacità di ingresso (un maggior numero di partizioni o di broker) o dall'adozione di un formato di deserializzazione più efficiente.

Studio del parallelismo su Flink — backlog pre-caricato (compute-bound)

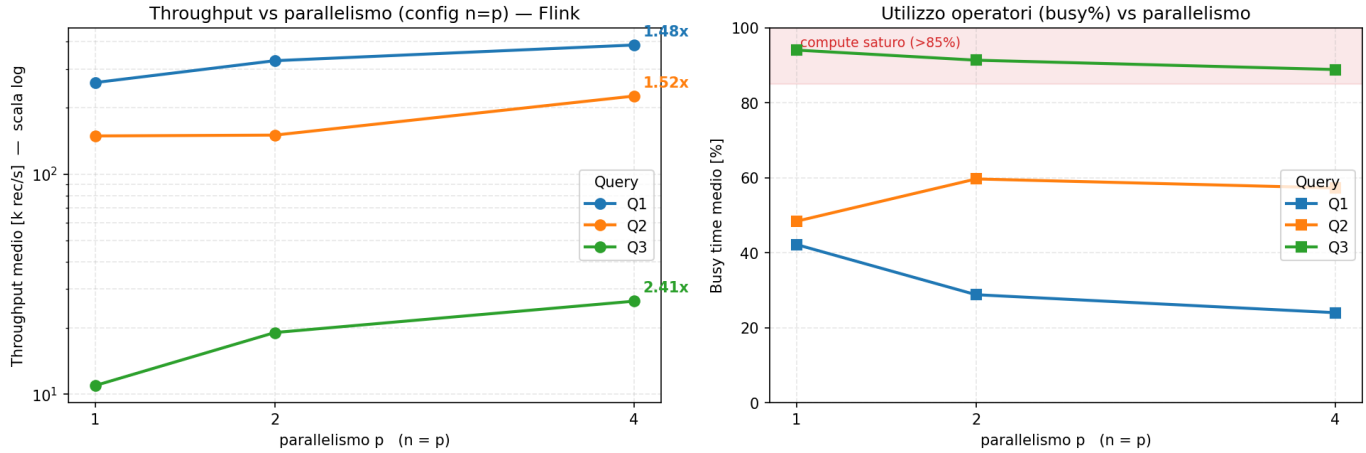


Figura 3. Throughput e Busyness rispetto alle configurazioni di parallelismo

Tabella VII
STUDIO DEL PARALLELISMO SU FLINK

Config (p, n)	Throughput [rec/s]	Elapsed [ms]	Busy Avg [%]	Speedup (vs p1_n1)
Q1 (1h)				
p1_n1	260846	8547	42.2	1.00×
p2_n2	328005	6797	28.8	1.26×
p4_n4	386665	5766	24.0	1.48×
Q2 (1h)				
p1_n1	149087	14954	48.4	1.00×
p2_n2	150354	14828	59.7	1.01×
p4_n4	226478	9844	57.3	1.52×
Q3 (1d)				
p1_n1	10911	204328	94.1	1.00×
p2_n2	19039	117094	91.4	1.75×
p4_n4	26369	84547	88.9	2.41×

X. USO DI STRUMENTI DI AI GENERATIVA PER IL PROGETTO 2

Durante lo svolgimento del *Progetto 2: Analisi in tempo reale di dati sui voli con Apache Flink* i principali strumenti di AI generativa utilizzati sono stati: *ChatGPT* e *GitHub Copilot*. L'impiego di questi strumenti ha avuto l'obiettivo di velocizzare lo sviluppo progetto, soprattutto nelle fasi che non richiedessero uno sviluppo di una qualsiasi logica o dell'applicazione di metodologie apprese durante il Corso di Sistemi e Architetture per Big Data, come per esempio la configurazione dei servizi o framework utilizzati. Precisiamo che l'architettura del progetto è stata totalmente sviluppata e ragionata dai membri del gruppo e che gli strumenti di AI generativa possono essere stati utilizzati nella fase progettuale solo con un'accezione esplorativa verso alternative possibili rispetto a quanto deciso in prima istanza, o per la ricerca di un'ottimizzazione delle configurazioni dell'architettura sviluppata da noi. Ogni scelta critica è stata presa dai membri del gruppo dopo un'attenta analisi di quanto generato

tramite questi strumenti e dopo un controllo ulteriore presso fonti ufficiali, quali le documentazioni dei servizi o framework utilizzati.

Gli strumenti sono stati utilizzati anche per una fase di pulizia del codice, in favore di una maggiore leggibilità dello stesso, andando a creare una ristrutturazione del codice che permettesse anche un riutilizzo di funzioni tra moduli diversi, sempre grazie a prompt specifici pensati dai membri del gruppo.

Nello sviluppo del codice ci siamo avvalsi dell'aiuto di strumenti di AI generativa per l'implementazione di algoritmi specifici, in particolare: *DDSketch* nel calcolo dei percentili nella Query 3. Ci teniamo a precisare che l'utilizzo di questi strumenti in questa fase è stato da supporto per un controllo ulteriore sulla correttezza delle implementazioni sviluppate, più che nello sviluppo delle stesse. Anche in questo caso, ogni scelta critica è stata presa dai membri del gruppo dopo un'attenta analisi di quanto generato tramite questi strumenti e dopo un controllo ulteriore presso fonti ufficiali, quali i riferimenti forniteci assieme alla specifica per alcuni di questi

algoritmi.

Per la fase di debugging durante lo sviluppo del progetto i membri del gruppo hanno ricorso agli strumenti di AI generativa, soprattutto nei casi in cui occorreva interpretare gli output di errori o warning restituiti dai servizi o framework utilizzati, in maniera tale da velocizzare il processo di individuazione dei problemi e permetterci quindi di agire in prima persona per la risoluzione degli stessi.

Il report presentato è stato scritto interamente dai membri del gruppo, senza avvalersi dell'utilizzo di strumenti di AI generativa per la stesura iniziale del testo. Non è avvenuto nemmeno un controllo sintattico e stilistico dei contenuti prodotti in maniera originale dai membri del gruppo, ma solo un controllo di coerenza del contenuto della relazione stessa con il codice sviluppato.

I membri del gruppo tengono a precisare di nuovo che ogni utilizzo degli strumenti generativi è stata utilizzata *da supporto* per lo sviluppo e non come *sostituto* dello stesso. Ogni riga di codice generata è stata analizzata e verificata prima di essere stata inserita manualmente all'interno del progetto.

RIFERIMENTI BIBLIOGRAFICI

- [1] C. Masson, J. E. Rim, and H. K. Lee, "DDSketch: A fast and fully-mergeable quantile sketch with relative-error guarantees," *Proc. VLDB Endowment*, vol. 12, no. 12, pp. 2195–2205, 2019.